
Introducing PyTorch



What is PyTorch?



Tensor library

What is PyTorch?



1 Tensor library

What is PyTorch?



1 **Tensor library**

2 **Automatic
differentiation engine**

What is PyTorch?



1 **Tensor library**

2 **Automatic
differentiation engine**

What is PyTorch?

3 **Deep learning
library**



What is PyTorch?

PyTorch is **free** and **open-source** software



History of PyTorch



PyTorch is Based on Torch



[https://en.wikipedia.org/wiki/Torch_\(machine_learning\)#/media/File:Torch_2014_logo.png](https://en.wikipedia.org/wiki/Torch_(machine_learning)#/media/File:Torch_2014_logo.png)



Torch was written in Lua



[https://en.wikipedia.org/wiki/Lua_\(programming_language\)#/media/File:Lua-Logo.svg](https://en.wikipedia.org/wiki/Lua_(programming_language)#/media/File:Lua-Logo.svg)



PyTorch Was Based on Torch 7

PYTORCH

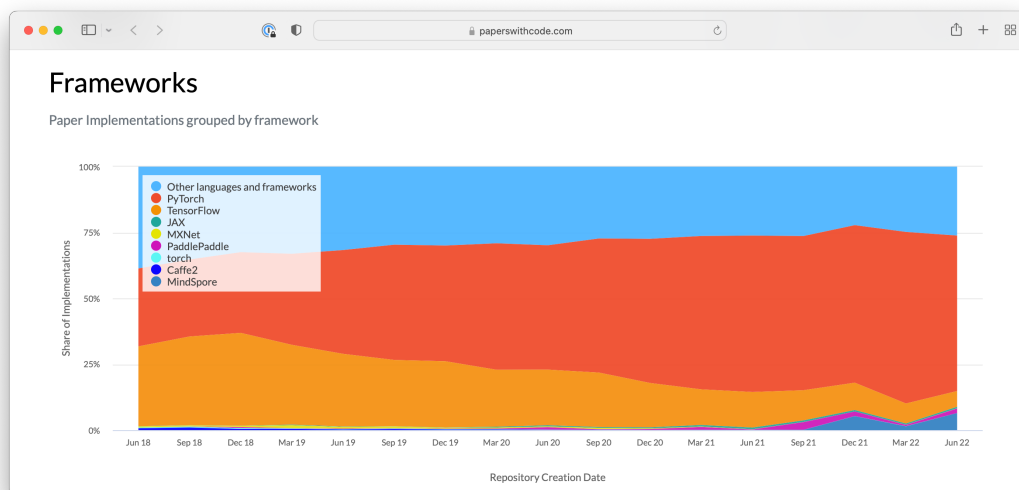
2016

PyTorch

Today



PyTorch is widely used



1 **Tensor library**

2 **Automatic
differentiation engine**

What is PyTorch?

3 **Deep learning
library**



What are tensors?

1 **Tensor library**

2 **Automatic
differentiation engine**

What is PyTorch?

3 **Deep learning
library**



What Are Tensors?

Part 1: Tensors for Data



1

Tensor library

2

Automatic
differentiation engine

What is PyTorch?

3

Deep learning
library



Tensor library



Tensor library

Mathematically: a generalization of vectors, matrices, etc.



Tensor library

Mathematically: a generalization of vectors, matrices, etc.

Computationally: a data container



Scalar (rank-0 tensor)



Scalar (rank-0 tensor)

In pure Python

```
a = 1.  
print(a)
```

1.0



Scalar (rank-0 tensor)

In pure Python

```
a = 1.  
print(a)
```

1.0

In PyTorch

```
import torch  
a = torch.tensor(1.)  
print(a)
```

tensor(1.)



Scalar (rank-0 tensor)

In pure Python

```
a = 1.  
print(a)
```

1.0

In PyTorch

```
import torch  
a = torch.tensor(1.)  
print(a)
```

tensor(1.)

```
print(a.shape)
```

torch.Size([1])



Vector (rank-1 tensor)



Vector (rank-1 tensor)

In pure Python

```
a = [1., 2., 3.]  
print(a)
```

[1.0, 2.0, 3.0]



Vector (rank-1 tensor)

In pure Python

```
a = [1., 2., 3.]  
print(a)
```

[1.0, 2.0, 3.0]

In PyTorch

```
a = torch.tensor([1., 2., 3.])  
print(a)
```

tensor([1., 2., 3.])



Vector (rank-1 tensor)

In pure Python

```
a = [1., 2., 3.]  
print(a)
```

[1.0, 2.0, 3.0]

In PyTorch

```
a = torch.tensor([1., 2., 3.])  
print(a)
```

tensor([1., 2., 3.])

```
print(a.shape)
```

torch.Size([3])



Matrix (rank-2 tensor)



Matrix (rank-2 tensor)

```
a = torch.tensor([[1., 2., 3.],  
                  [2., 3., 4.]])  
a.shape  
torch.Size([2, 3])
```



Matrix (rank-2 tensor)

Where have we seen a structure like this before?

```
a = torch.tensor([[1., 2., 3.],  
                  [2., 3., 4.]])  
a.shape  
torch.Size([2, 3])
```



Matrix (rank-2 tensor)

Where have we seen a structure like this before?

```
a = torch.tensor([[1., 2., 3.],  
                  [2., 3., 4.]])  
a.shape  
torch.Size([2, 3])
```

← Training example



Matrix (rank-2 tensor)

Where have we seen a structure like this before?

```
a = torch.tensor([[1., 2., 3.],  
                  [2., 3., 4.]])  
a.shape  
torch.Size([2, 3])
```

Feature



RGB Image



Image Source: <https://code.tutsplus.com/tutorials/create-a-retro-art-distortion-effect-using-rgb-shifting-active-3369>



RGB Image

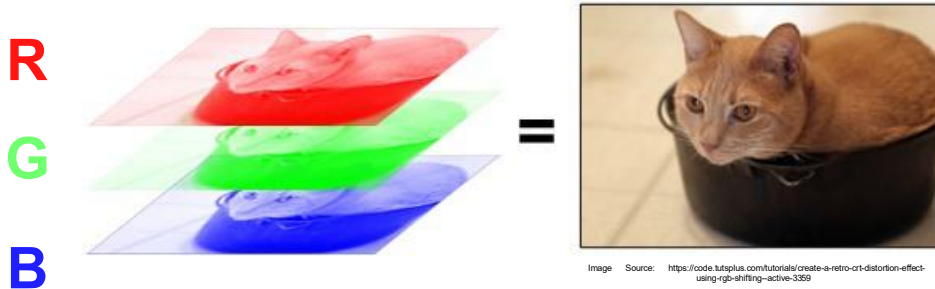
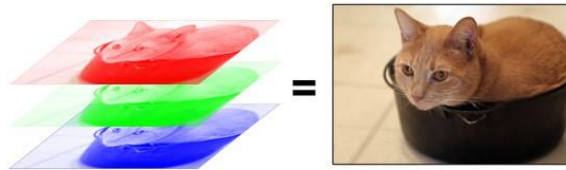


Image Source: <https://code.tutsplus.com/tutorials/create-a-retro-art-distortion-effect-using-rgb-shifting-active-3369>

Color image as a stack of matrices



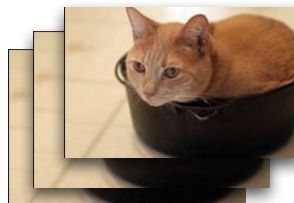


Color image as a stack of matrices

3D tensor
(rank-3 tensor)

```
a = torch.tensor([[[1., 2., 3.],  
                  [2., 3., 4.]],  
                  [[5., 6., 7.],  
                  [8., 9., 10.]])  
a.shape
```

```
[6]:  
torch.Size([2, 2, 3])
```



A stack of multiple color images





4D tensor
(rank-4 tensor)

```
b = torch.stack((a, a))  
b.shape
```

```
torch.Size([2, 2, 2, 3])
```

```
b
```

```
tensor([[[[ 1.,  2.,  3.],  
          [ 2.,  3.,  4.]],  
        [[ 5.,  6.,  7.],  
          [ 8.,  9., 10.]]],
```

```
        [[[ 1.,  2.,  3.],  
          [ 2.,  3.,  4.]],
```

```
        [[ 5.,  6.,  7.],
```



Tensor library Vs. Array library



What Are Tensors?

Part 2: Tensors and Array Libraries



1

Tensor library

2

**Automatic
differentiation engine**

What is PyTorch?

3

**Deep learning
library**



Tensor library

Mathematically: a generalization of vectors, matrices, etc.



Tensor library

Tensor library = array library



Tensor library

torch.tensor numpy.array



Tensor library

torch.tensor numpy.array

+GPU support



Tensor library

torch.tensor numpy.array

+GPU support

+automatic differentiation support



Python Lists Vs. Arrays/Tensors



Python Lists Vs. Arrays/Tensors

List



Python Lists Vs. Arrays/Tensors

List

- + Can store heterogenous types
(mix str, float, etc.)
- + Elements can be easily added or removed



Python Lists Vs. Arrays/Tensors

List

- + Can store heterogenous types (mix str, float, etc.)
- + Elements can be easily added or removed
- Numerical computations are slow



Python Lists Vs. Arrays/Tensors

List

Tensor

- + Can store heterogenous types (mix str, float, etc.)
- + Elements can be easily added or removed
- Numerical computations are slow



Python Lists Vs. Arrays/Tensors

List

- + Can store heterogenous types (mix str, float, etc.)
- + Elements can be easily added or removed
- Numerical computations are slow



Tensor

- All elements have to have the same type (e.g., int, float)
- Can't add or remove elements

Python Lists Vs. Arrays/Tensors

List

- + Can store heterogenous types (mix str, float, etc.)
- + Elements can be easily added or removed
- Numerical computations are slow



Tensor

- All elements have to have the same type (e.g., int, float)
- Can't add or remove elements
- + Supports many mathematical operations
- + Numerical computations are fast

How Do We Use Tensors in PyTorch?



How Do We Use Tensors in PyTorch?



Top 10 Tensor Functions and Methods



Top 10 Tensor Functions and Methods



1

Creating Tensors

```
import torch

m = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])
print(v)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```



2

Checking the Shape

```
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

[55]:

```
print(m.shape)
```

```
torch.Size([2, 3])
```



3

Checking the Rank / Number of Dimensions

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.ndim)
```

```
2
```



4

Checking the Data Type

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.dtype)
```

```
torch.float32
```



4

Checking the Data Type

```
other_m = torch.tensor([[1, 2, 3],  
                        [4, 5, 6]])  
print(other_m)
```

```
tensor([[1, 2, 3],  
        [4, 5, 6]])
```



4

Checking the Data Type

```
other_m = torch.tensor([[1, 2, 3],  
                        [4, 5, 6]])  
print(other_m)
```

```
tensor([[1, 2, 3],  
        [4, 5, 6]])
```

```
print(other_m.dtype)
```

```
torch.int64
```



5

Creating a Tensor From NumPy Arrays

```
import numpy as np

np_ary = np.array([1., 2., 3.])
m2 = torch.from_numpy(np_ary)

print(m2)
```

```
tensor([1., 2., 3.], dtype=torch.float64)
```



5

Creating a Tensor From NumPy Arrays

```
import numpy as np

np_ary = np.array([1., 2., 3.])
m2 = torch.from_numpy(np_ary)

print(m2)
```

```
tensor([1., 2., 3.], dtype=torch.float64)
```



5

Creating a Tensor From NumPy Arrays

```
import numpy as np
np_ary = np.array([1., 2., 3.])
m2 = torch.from_numpy(np_ary)
print(m2)

tensor([1., 2., 3.], dtype=torch.float64)
```

```
m3 = torch.tensor(np_ary)
print(m3)
```



tensor([1., 2., 3.], dtype=torch.float64)

6

Changing Data Types

```
print(m2.dtype)
```

torch.float64



6

Changing Data Types

```
print(m2.dtype)
```

```
torch.float64
```

```
[47]:
```

```
m2 = m2.to(torch.float32)  
print(m2.dtype)
```

```
torch.float32
```



7

Checking the Device Type

```
m2.device
```

```
device(type='cpu')
```



8

Changing the Tensor Shape

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.view(3, 2))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```



8

Changing the Tensor Shape

```
print(m.view(3, 2))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```

[69]:

```
print(m.view(-1, 2))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```



8

Changing the Tensor Shape

```
print(m.view(3, 2))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```

[69]:

```
print(m.view(-1, 2))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```

[70]:

```
print(m.view(3, -1))
```

```
tensor([[1., 2.],  
        [3., 4.],  
        [5., 6.]])
```



9

Transposing a Tensor

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.T)
```

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```



9

Transposing a Tensor

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

1.	2.	3.
4.	5.	6.

```
print(m.T)
```

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```



9

Transposing a Tensor

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.T)
```

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```

1.	4.
2.	5.
3.	6.



10

Multiplying Matrices

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```



10

Multiplying Matrices

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.T)
```

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```



10

Multiplying Matrices

```
print(m)
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

```
print(m.T)
```

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```

```
m.matmul(m.T)
```

```
[73]:
```

```
tensor([[14., 32.],  
        [32., 77.]])
```



Using concepts from linear algebra to make our code more efficient ...



4

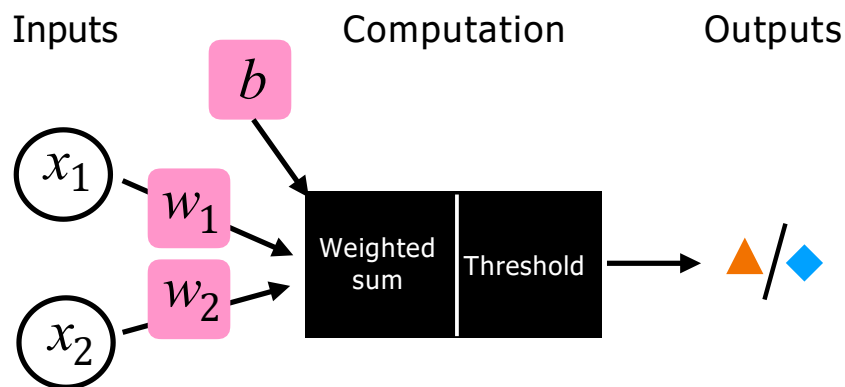
Improving Code Efficiency with Linear Algebra

Part 1: From For-Loops to Dot Products

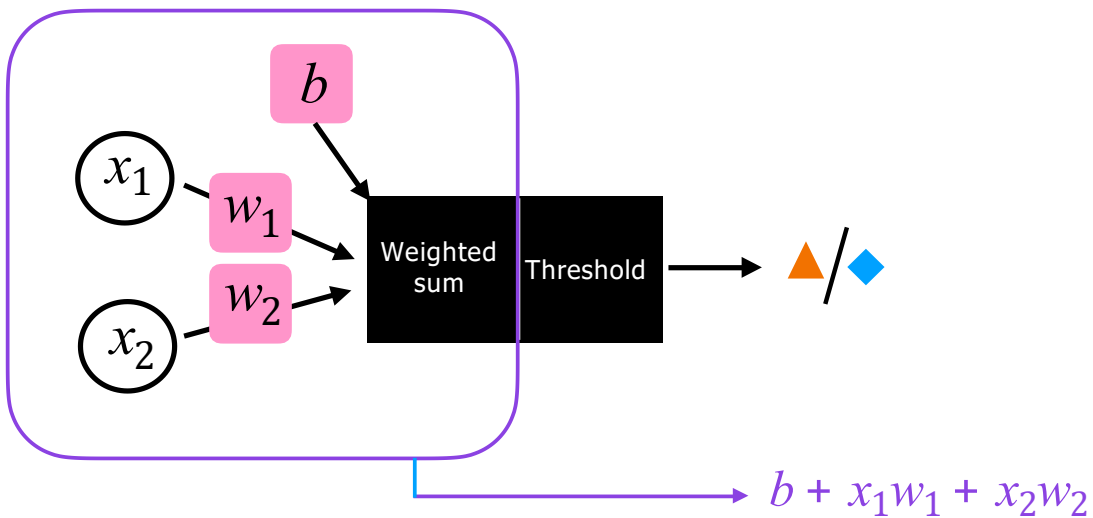


Sebastian Raschka and the Lightning AI Team

The Perceptron



The Perceptron



1 training example
with 2 features values

$$z = b + x_1 \times w_1 + x_2 \times w_2$$



1 training example
with 2 features values

$$z = b + w_1 \times x_1 + w_2 \times x_2$$

General form

$$z = b + \sum_{j=1}^m x_j w_j$$



Weighted Sum in Plain Python

$$z = b + \sum_{j=1}^m x_j w_j$$

```
b = 0.  
x = [1.2, 2.2]  
w = [3.3, 4.3]  
  
output = b  
for x_j, w_j in zip(x, w):  
    output += x_j * w_j  
  
print(output)
```



Weighted Sum in PyTorch using Linear Algebra

$$z = b + \sum_{j=1}^m x_j w_j$$

$$= b + \mathbf{xw}$$

Dot product between two vectors



Dot Product Between 2 Vectors

$$z = b + \mathbf{xw}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$



Dot Product Between 2 Vectors

$$z = b + \mathbf{x}^\top \mathbf{w}$$

Transpose $^\top$ not required,
but more intuitive when we
talk about matrix multiplication later

$$\mathbf{x}^\top = [x_1 \ x_2 \ \dots \ x_m]$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$



Dot Product Between 2 Vectors

$$z = b + \mathbf{xw}$$

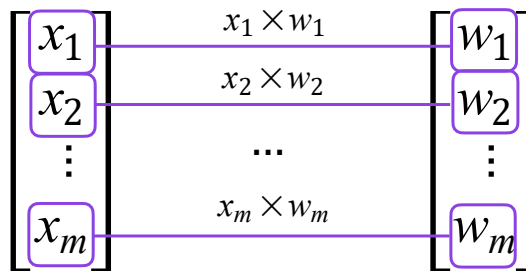
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$



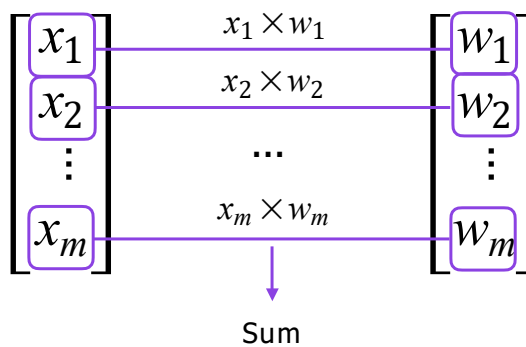
Dot Product Between 2 Vectors

$$z = b + \mathbf{xw}$$



Dot Product Between 2 Vectors

$$z = b + \mathbf{xw}$$



Dot Product in PyTorch

```
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

tensor([13.4200])



Dot Product in PyTorch

```
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

tensor([13.4200])

```
b = 0.
x = [1.2, 2.2]
w = [3.3, 4.3]

output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j

print(output)
```

13.42



Dot product is more compact, but is it also faster?



Dot Product in PyTorch

```
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

```
13.42
tensor([13.4200])
```

```
b = 0.
x = [1.2, 2.2]
w = [3.3, 4.3]

output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j

print(output)
```

```
13.42
```



Dot Product in PyTorch

It is more compact, but is it also faster?

```
output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j
```

```
output = x.dot(w) + b
```



Benchmarking the Plain PyTorch Approach

```
def plain_python(x, w, b):
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    return output
```



Benchmarking the Plain Python Approach

```
def plain_python(x, w, b):  
    output = b  
    for x_j, w_j in zip(x, w):  
        output += x_j * w_j  
    return output
```

```
import random
```

```
random.seed(123)
```

```
b = 0.
```

```
x = [random.random() for _ in range(1000)]
```

```
w = [random.random() for _ in range(1000)]
```



Benchmarking the Plain Python Approach

```
def plain_python(x, w, b):  
    output = b  
    for x_j, w_j in zip(x, w):  
        output += x_j * w_j  
    return output
```

```
import random
```

```
random.seed(123)
```

```
b = 0.
```

```
x = [random.random() for _ in range(1000)]
```

```
w = [random.random() for _ in range(1000)]
```

```
%timeit plain_python(x, w, b)
```

35.3 μ s $\pm$ 175 ns per loop (mean $\pm$ std. dev. of
7 runs. 10.000 loops each)



Benchmarking the Dot Product

```
def pytorch_dot(x, w, b):  
    return x.dot(w) + b
```



Benchmarking the Dot Product

```
def pytorch_dot(x, w, b):  
    return x.dot(w) + b
```

```
t_b = torch.tensor(b)  
t_x = torch.tensor(x)  
t_w = torch.tensor(w)
```



Benchmarking the Dot Product

```
def pytorch_dot(x, w, b):  
    return x.dot(w) + b
```

```
t_b = torch.tensor(b)  
t_x = torch.tensor(x)  
t_w = torch.tensor(w)
```

```
%timeit pytorch_dot(t_x, t_w, t_b)
```

1.42 μ s $\pm$ 18.6 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)



Benchmarking the Dot Product

A 25x speed-up!

```
%timeit pytorch_dot(t_x, t_w, t_b)
```

1.42 μ s $\pm$ 18.6 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

```
%timeit plain_python(x, w, b)
```

35.3 μ s $\pm$ 175 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)



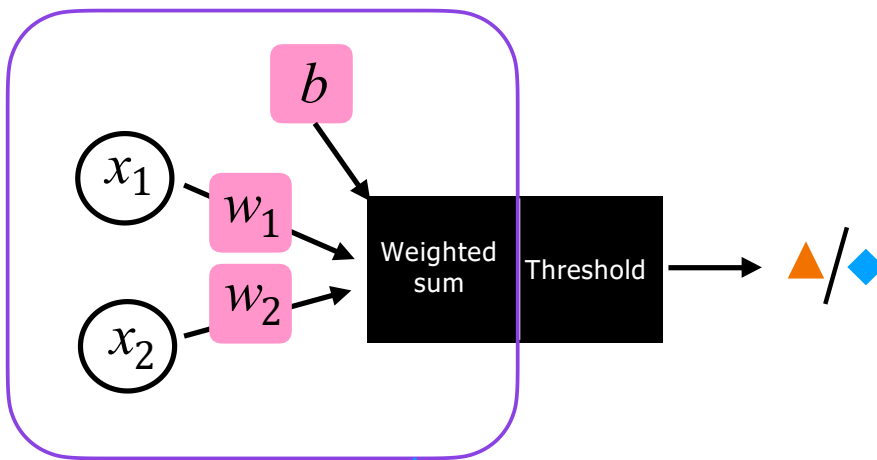
What about multiple training examples?
Let's have a look at matrix multiplication!



Improving Code Efficiency with Linear Algebra
Part 2: Dealing with Multiple Training Examples via Matrix Multiplication



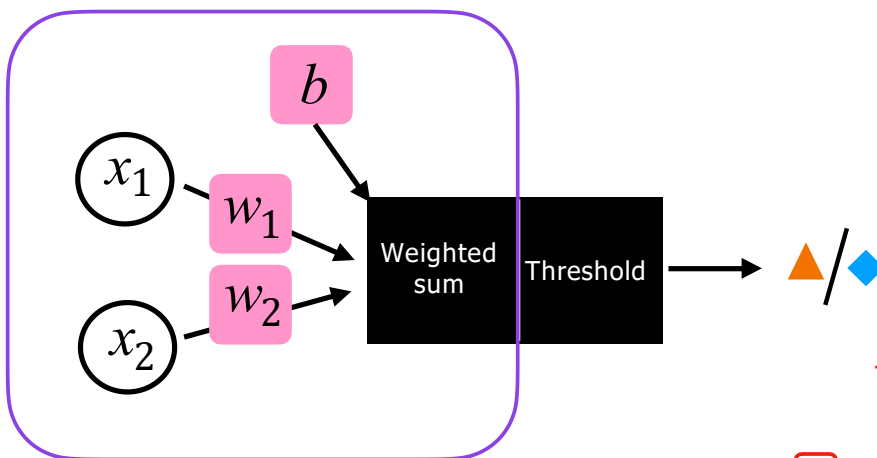
The Perceptron



$$z = b + \sum_{j=1}^m x_j w_j$$



The Perceptron



1 training example

$$z^{[i]} = b + \sum_{j=1}^m x_j w_j^{[i]}$$



Processing Multiple Training Examples

$$z^{[1]} = b + \sum_{j=1}^m x_j w_j^{[1]}$$

$$z^{[2]} = b + \sum_{j=1}^m x_j w_j^{[2]}$$

• • •

$$z^{[n]} = b + \sum_{j=1}^m x_j w_j^{[n]}$$



Processing Multiple Training Examples

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

• • •

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$



Training Examples as a Matrix

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

• • •

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$



Training Examples as a Matrix

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

• • •

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{[1]} \\ \mathbf{x}^{[2]} \\ \vdots \\ \mathbf{x}^{[n]} \end{bmatrix}$$



Training Examples as a Matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{[1]} \\ \mathbf{x}^{[2]} \\ \vdots \\ \mathbf{x}^{[n]} \end{bmatrix} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ & & \vdots & \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix}$$



Training Examples as a Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ & & \vdots & \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$



training examples

1 weight vector

Matrix Multiplication

Requirement: Number of columns and rows have to match

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$



training examples

1 weight vector

Matrix Multiplication

Dot product

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \quad \mathbf{Xw} = \begin{bmatrix} \boxed{} \\ \\ \\ \end{bmatrix}$$



training examples

1 weight vector

Result

Matrix Multiplication

Dot product

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \quad \mathbf{Xw} = \begin{bmatrix} \boxed{} \\ \boxed{} \\ \vdots \\ \boxed{} \end{bmatrix}$$

 training examples 1 weight vector Result

Matrix Multiplication

Dot product

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \quad \mathbf{Xw} = \begin{bmatrix} \boxed{} \\ \boxed{} \\ \vdots \\ \boxed{} \end{bmatrix}$$

 training examples 1 weight vector Result

Benchmark Time!



Multiplying a matrix and a vector in plain Python

```
b = 0.
X = [[1.2, 2.2],
     [4.4, 5.5]]
w = [3.3, 4.3]

outputs = []
for x in X:
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    outputs.append(output)

outputs
```



```
[13.42, 38.17]
```

Multiplying a matrix and a vector in PyTorch

```
b = 0.
X = [[1.2, 2.2],
      [4.4, 5.5]]
w = [3.3, 4.3]

outputs = []
for x in X:
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    outputs.append(output)

outputs
```

[13.42, 38.17]

```
import torch

b = torch.tensor([0.])
X = torch.tensor(
    [[1.2, 2.2],
     [4.4, 5.5]]
)
w = torch.tensor([3.3, 4.3])

X.matmul(w) + b

tensor([13.4200, 38.1700])
```



Benchmarking the Plain Python implementation

```
import random

random.seed(123)

b = 0.
X = [[random.random() for _ in range(1000)] # 500 rows
      for i in range(500)]
w = [random.random() for _ in range(1000)]

def plain_python(X, w, b):
    outputs = []
    for x in X:
        output = b
        for x_j, w_j in zip(x, w):
            output += x_j * w_j
        outputs.append(output)
    return outputs
```

```
%timeit plain_python(X, w, b)

19 ms ± 205 µs per loop (mean ± std. dev. of 7 runs, 100
```



Benchmarking the PyTorch implementation

```
t_b = torch.tensor(b)
t_X = torch.tensor(X)
t_w = torch.tensor(w)
```

```
def pytorch_implementation(X, w, b):
    return X.matmul(w) + b
```

```
%timeit pytorch_implementation(t_X, t_w, t_b)
```

23 μ s $\pm$ 1.64 μ s per loop (mean $\pm$ std. dev. of 7 runs,
10,000 loops each)



approx. 1000x speed-up!

Next: Matrix multiplication with 2 matrices



Improving Code Efficiency with Linear Algebra

Part 3: Multiplying Two Matrices



Matrix Multiplication with a Weight Matrix



Matrix Multiplication with a Weight Matrix

Requirement: Number of columns and rows have to match

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, & \dots & , x_m^{[2]} \\ & & \vdots & \\ x_1^{[n]} & x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} w_1^{(1)} & w_1^{(1)}, & \dots & , w_m^{(1)} \\ w_1^{(2)} & w_2^{(2)}, & \dots & , w_m^{(2)} \\ & & \vdots & \\ w_1^{(h)} & w_2^{(h)}, & \dots & , w_m^{(h)} \end{bmatrix}$$



training examples

weight vectors

Matrix Multiplication with a Weight Matrix

Requirement: Number of columns and rows have to match

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, & \dots & , x_m^{[2]} \\ & & \vdots & \\ x_1^{[n]} & x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^T = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, & \dots & , w_2^{(h)} \\ & & \vdots & \\ w_m^{(1)} & w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix}$$



training examples

weight vectors

Matrix Multiplication with Weight Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^\top = \begin{bmatrix} \boxed{} & & \\ & & \\ & & \\ & & \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^\top = \begin{bmatrix} \boxed{} & & \\ \boxed{} & & \\ & & \\ & & \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

Diagram illustrating the calculation of an element in the resulting matrix $\mathbf{XW}^T$. The input matrix $\mathbf{X}$ is shown as a matrix of rows, with the i -th row highlighted. The weight matrix $\mathbf{W}^T$ is shown as a matrix of columns, with the j -th column highlighted. The resulting matrix $\mathbf{XW}^T$ is shown as a matrix of rows, with the j -th column highlighted. The diagram shows the dot product of the i -th row of $\mathbf{X}$ and the j -th column of $\mathbf{W}^T$ to calculate the element in the j -th column of $\mathbf{XW}^T$.

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^T = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^T = \begin{bmatrix} c_1^{(1)} & c_2^{(1)}, \dots, c_h^{(1)} \\ c_1^{(2)} & c_2^{(2)}, \dots, c_h^{(2)} \\ \vdots & \vdots \\ c_1^{(n)} & c_2^{(n)}, \dots, c_h^{(n)} \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

Diagram illustrating the calculation of an element in the resulting matrix $\mathbf{XW}^T$. The input matrix $\mathbf{X}$ is shown as a matrix of rows, with the i -th row highlighted. The weight matrix $\mathbf{W}^T$ is shown as a matrix of columns, with the j -th column highlighted. The resulting matrix $\mathbf{XW}^T$ is shown as a matrix of rows, with the j -th column highlighted. The diagram shows the dot product of the i -th row of $\mathbf{X}$ and the j -th column of $\mathbf{W}^T$ to calculate the element in the j -th column of $\mathbf{XW}^T$.

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^T = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^T = \begin{bmatrix} c_1^{(1)} & c_2^{(1)}, \dots, c_h^{(1)} \\ c_1^{(2)} & c_2^{(2)}, \dots, c_h^{(2)} \\ \vdots & \vdots \\ c_1^{(n)} & c_2^{(n)}, \dots, c_h^{(n)} \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^\top = \begin{bmatrix} \boxed{}^{(1)} & \boxed{}^{(2)} & \dots & \boxed{}^{(h)} \\ \boxed{} & \boxed{} & \dots & \boxed{} \\ \vdots & \vdots & \ddots & \vdots \\ \boxed{} & \boxed{} & \dots & \boxed{} \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix} \quad \mathbf{XW}^\top = \begin{bmatrix} \boxed{}^{(1)} & \boxed{}^{(2)} & \dots & \boxed{}^{(h)} \\ \boxed{} & \boxed{} & \dots & \boxed{} \\ \vdots & \vdots & \ddots & \vdots \\ \boxed{} & \boxed{} & \dots & \boxed{} \end{bmatrix}$$



Matrix Multiplication with Weight Matrix

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix}$$

$\mathbf{XW}^\top =$



Output Dimensions

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_1^{[1]}, \dots, x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]}, \dots, x_m^{[2]} \\ \vdots & \vdots \\ x_1^{[n]} & x_2^{[n]}, \dots, x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} & w_1^{(2)}, \dots, w_1^{(h)} \\ w_2^{(1)} & w_2^{(2)}, \dots, w_2^{(h)} \\ \vdots & \vdots \\ w_m^{(1)} & w_m^{(2)}, \dots, w_m^{(h)} \end{bmatrix}$$

$\mathbf{XW}^\top =$



A PyTorch Example

```
import torch

X = torch.rand(100, 10)
W = torch.rand(50, 10)

R = torch.matmul(X, W.T)
```

```
R.shape
```

```
torch.Size([100, 50])
```



Broadcasting — Computations with Unequal Tensor Shapes



Improving Code Efficiency with Linear Algebra

Part 4: Broadcasting — Computations with Unequal Tensor Shapes



$$\begin{bmatrix} 1.1 & 2.1 & 3.1 & 4.1 \end{bmatrix} + \begin{bmatrix} 5.6 \end{bmatrix}$$



$$\begin{array}{|c|c|c|c|} \hline 1.1 & 2.1 & 3.1 & 4.1 \\ \hline \end{array} + \begin{array}{|c|c|c|c|} \hline 5.6 & 5.6 & 5.6 & 5.6 \\ \hline \end{array}$$



$$\begin{array}{|c|c|c|c|} \hline 1.1 & 2.1 & 3.1 & 4.1 \\ \hline \end{array} + \begin{array}{|c|c|c|c|} \hline 5.6 & 5.6 & 5.6 & 5.6 \\ \hline \end{array}$$

$$= \begin{array}{|c|c|c|c|} \hline 6.7 & 7.7 & 8.7 & 9.7 \\ \hline \end{array}$$



$$\begin{bmatrix} 1.1 & 2.1 & 3.1 & 4.1 \end{bmatrix} + \begin{bmatrix} 5.6 & 5.6 & 5.6 & 5.6 \end{bmatrix} = \begin{bmatrix} 6.7 & 7.7 & 8.7 & 9.7 \end{bmatrix}$$

```
a = torch.tensor([1.1, 2.1, 3.1, 4.1])  
b = torch.tensor([5.6])  
  
a + b
```

```
tensor([6.7000, 7.7000, 8.7000, 9.7000])
```



$$\begin{bmatrix} 1.1 & 2.1 & 3.1 & 4.1 \\ 1.2 & 2.2 & 3.2 & 4.2 \end{bmatrix} + \begin{bmatrix} 5.4 & 5.5 & 5.6 & 5.7 \end{bmatrix}$$



1.1	2.1	3.1	4.1
1.2	2.2	3.2	4.2

 +

5.4	5.5	5.6	5.7
5.4	5.5	5.6	5.7



1.1	2.1	3.1	4.1
1.2	2.2	3.2	4.2

 +

5.4	5.5	5.6	5.7
5.4	5.5	5.6	5.7

```
A = torch.tensor([[1.1, 2.1, 3.1, 4.1],  
                  [1.2, 2.2, 3.2, 4.2]])
```

```
b = torch.tensor([5.4, 5.5, 5.6, 5.7])
```

```
A + b
```

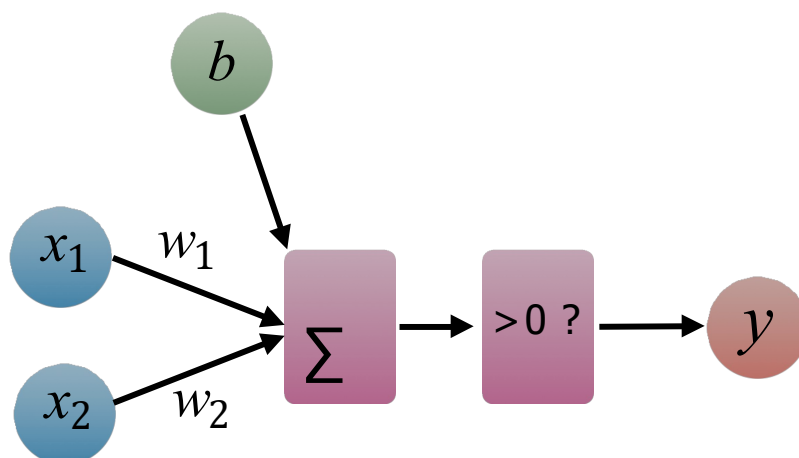
```
tensor([[6.5000, 7.6000, 8.7000, 9.8000],  
        [6.6000, 7.7000, 8.8000, 9.9000]])
```



Seeing Predictive Models as Computation Graphs



The Perceptron Model



```
class Perceptron:
    def __init__(self, num_features):
        self.num_features = num_features
        self.weights = torch.zeros(num_features)
        self.bias = torch.tensor(0.)

    def forward(self, x):
        weighted_sum_z = torch.dot(x, self.weights) + self.bias

        if weighted_sum_z > 0.:
            prediction = torch.tensor(1.)
        else:
            prediction = torch.tensor(0.)

        return prediction

    def update(self, x, true_y):
        prediction = self.forward(x)
        error = true_y - prediction

        # update
        self.bias += error
        self.weights += error * x

        return error
```



Model Parameters

b

w_1

w_2



Data Inputs

b

x_1

w_1

x_2



w_2

Weighted Sum

b

x_1



$$a = w_1 \times x_1$$

w_1

x_2

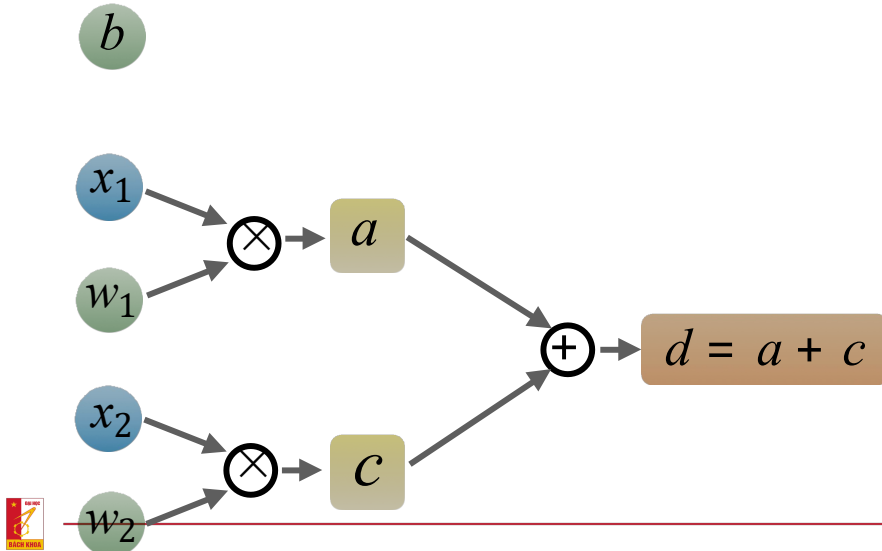


$$b = w_2 \times x_2$$

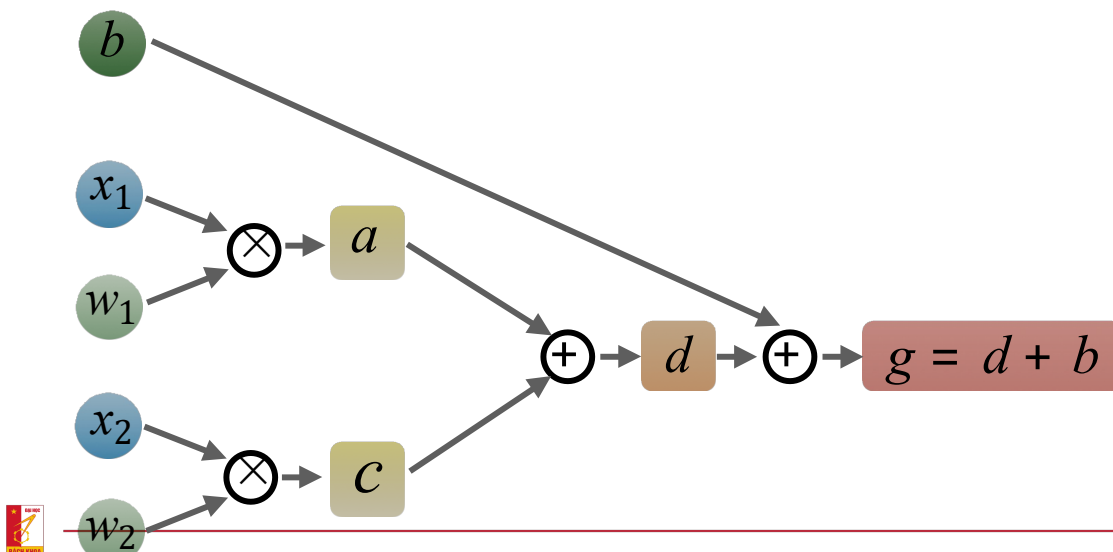


w_2

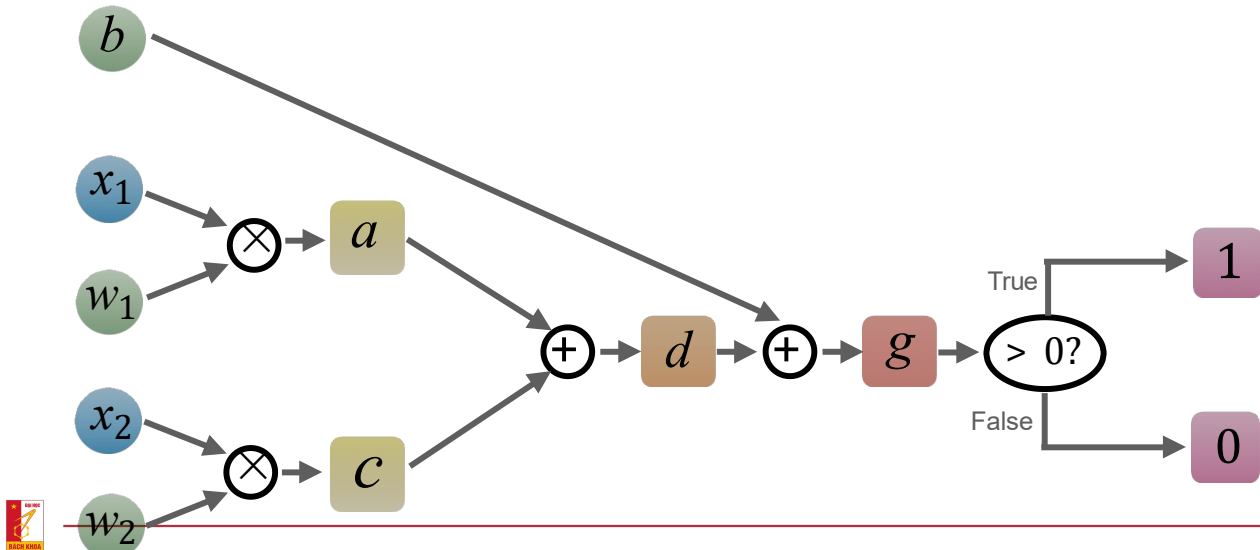
Weighted Sum



Weighted Sum



Threshold Function



```
class Perceptron:
    def __init__(self, num_features):
        self.num_features = num_features
        self.weights = torch.zeros(num_features)
        self.bias = torch.tensor(0.)

    def forward(self, x):
        weighted_sum_z = torch.dot(x, self.weights) + self.bias

        if weighted_sum_z > 0.:
            prediction = torch.tensor(1.)
        else:
            prediction = torch.tensor(0.)

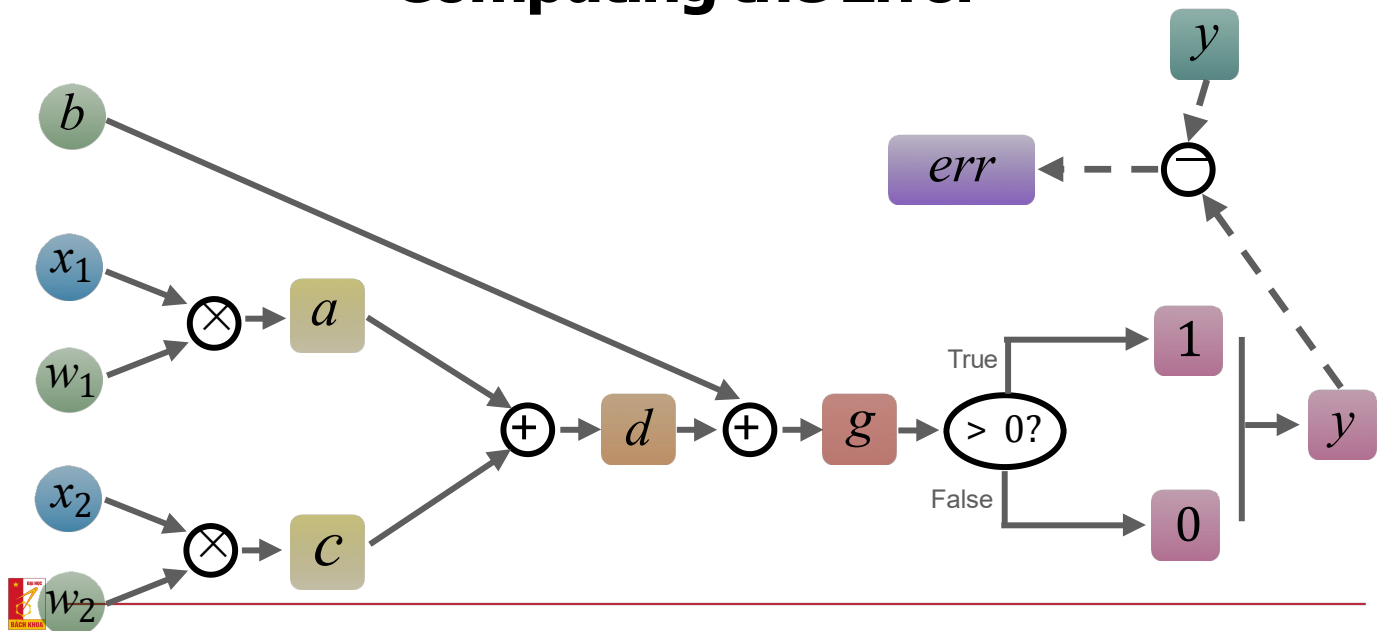
        return prediction

    def update(self, x, true_y):
        prediction = self.forward(x)
        error = true_y - prediction

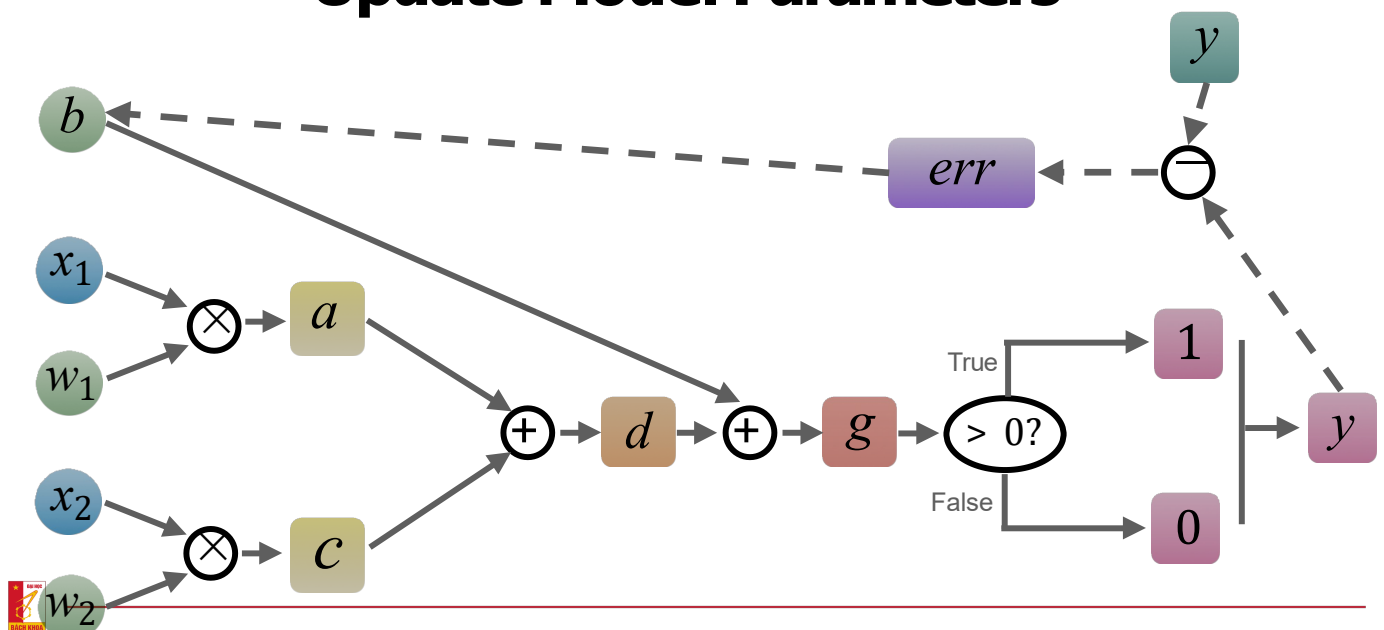
        # update
        self.bias += error
        self.weights += error * x

        return error
```

Computing the Error



Update Model Parameters



Update Model Parameters

